

API Integratie voor Odoo en andere ERP systemen

REALISATIEDOCUMENT

Brent Paessens
Student Bachelor in de Toegepaste Informatica – Applicatieontwikkeling
Stagegever: Vervio
Stageperiode: februari – mei 2026

Inhoudsopgave

1. INLEIDING	3
2. ANALYSE	4
2.1. Inleiding	4
2.2. Overzicht van onderzochte oplossingen	4
2.3. Vergelijking van de tools	5
2.4. Weighted Ranking Methode	5
2.5. Gemaakte keuze	6
2.6. Keuze van programmeertaal: Python vs. C#	6
2.7. Besluit van de analyse	6
3. SYNCHRONISATIE VAN PRODUCTEN	7
3.1. Productsynchronisatie: Odoo → WooCommerce	7
3.2. Merken (brands)	9
4. ORDER TERUGKOPPELING	10
4.1. Ordersynchronisatie: WooCommerce → Odoo	10
5. PLANNING	12
6. BESLUIT	13
LITERATUURLIJST	15
BIJLAGEN	16
TIPS VOOR TEKSTEN IN (DOCUMENTEN OP) JE AFSTUDEERPORTFOLIO FOUT! BLADWIJZER NIET GEDEFINIEERD.	

1. Inleiding

Dit document beschrijft hoe het stageproject bij Vervio, een bedrijf dat zich richt op digitale integraties voor kleine en middelgrote ondernemingen (kmo's) met behulp van het ERP-platform Odoo, in de praktijk werd uitgevoerd. De stageperiode liep van februari tot en met mei 2026 en werd begeleid door een technische mentor vanuit Vervio. Waar projectplan de nadruk legt op de doelstellingen en de geplande aanpak, richt dit realisatiedocument zich op de concrete uitvoering: welke keuzes gemaakt werden, waarom en hoe het eindresultaat tot stand is gekomen.

Het project heeft als doel een automatische verbinding te bouwen tussen twee systemen die in veel bedrijven afzonderlijk werken: Odoo het Enterprise Resource Planning (ERP)-systeem waarin producten en bestellingen beheerd worden, en WordPress met WooCommerce, de online webshop. Concreet houdt dit in dat productinformatie vanuit Odoo automatisch naar de webshop wordt gestuurd en dat bestellingen die klanten plaatsen in de webshop automatisch terugvloeien naar Odoo. Dit elimineert handmatig werk en vermindert de kans op fouten.

Dit document is als volgt opgebouwd: Hoofdstuk 2 beschrijft de voorafgaande analyse die aan de basis ligt van de technische keuzes. Hoofdstuk 3 licht de werking van de productsynchronisatie toe. Hoofdstuk 4 beschrijft de automatische orderverwerking. Hoofdstuk 5 biedt een overzicht van de weekplanning en hoofdstuk 6 sluit af met een besluit, een evaluatie van de doelstellingen en aanbevelingen voor toekomstige ontwikkeling. De bijlagen bevatten aanvullend materiaal zoals de detailberekening van de keuzemethode, een architectuuroverzicht, relevante codefragmenten en een lijst van gebruikte afkortingen.

2. Analyse

2.1. Inleiding

Alvorens met de implementatie te starten, werd grondig onderzocht welke aanpak het beste aansluit bij de vereisten van het project. Dit hoofdstuk beschrijft hoe die analyse verliep en tot welke conclusies zij leidde.

Op basis van het projectplan kwamen vier concrete vereisten naar voren waaraan de oplossing diende te voldoen:

- De productgegevens moeten automatisch gesynchroniseerd worden, zonder handmatige tussenkomst.
- Bestellingen moeten betrouwbaar en foutloos verwerkt worden.
- De oplossing moet flexibel genoeg zijn om later uit te breiden naar andere platformen of functies.
- De totale kost op lange termijn moet beperkt blijven.

2.2. Overzicht van onderzochte oplossingen

Drie mogelijke aanpakken werden onderzocht en tegenover elkaar afgewogen:

No-code tools (tools zonder programmeerkennis)

- **Zapier**: een populaire tool waarmee koppelingen tussen applicaties gemaakt kunnen worden zonder code.
- **n8n**: een gelijkaardig platform dat ook zelf gehost kan worden, wat meer controle biedt.

Maatwerkontwikkeling

- Een Python-script op maat dat rechtstreeks communiceert met de Application Programming Interfaces (API's) van Odoo en WooCommerce.

In dit project worden de API's van Odoo B.V. (z.d.-c) en WordPress.org (z.d.) gebruikt om gegevens automatisch uit te wisselen.

2.3. Vergelijking van de tools

De drie oplossingen werden beoordeeld op vijf criteria: kostprijs, flexibiliteit, schaalbaarheid, gebruiksgemak en technische controle. Hierna volgt een samenvatting van de bevindingen.

- Zapier is snel op te zetten en eenvoudig in gebruik, maar heeft duidelijke beperkingen bij complexe logica. Bovendien loopt de kostprijs bij intensief gebruik snel op.
- n8n biedt meer vrijheid en kan gratis zelf gehost worden, waardoor de kosten op lange termijn lager liggen. Er is echter wel technische kennis nodig om het te beheren.
- Een Python-script op maat vergt meer ontwikkeltijd, maar biedt maximale controle en kan volledig afgestemd worden op de specifieke noden van het project.

2.4. Weighted Ranking Methode

Om een objectieve en onderbouwde keuze te maken, werd de Weighted Ranking Methode toegepast. Bij deze methode krijgt elk criterium een gewicht op basis van zijn belang voor het project. Vervolgens krijgt elke oplossing een score per criterium en wordt een gewogen totaal berekend. De volledige detailberekening is terug te vinden in Bijlage A.

Criteria	Gewicht
Kostprijs	30%
Flexibiliteit	25%
Schaalbaarheid	20%
Gebruiksgemak	15%
Technische controle	10%

De scores per oplossing zien er als volgt uit (score op 5):

Oplossing	Kost	Flex	Schaal	Gebruik	Controle	Totaal
Zapier	2	2	3	4	1	2.40
n8n	2	3	3	3	3	2.80
Python script	5	5	5	2	5	4.40

2.5. Gemaakte keuze

Op basis van de analyse en de gewogen scores werd er gekozen voor een Python-script op maat. Hoewel deze aanpak meer ontwikkeltijd vereist dan de no-code alternatieven, biedt zij een aantal cruciale voordelen:

- Volledige vrijheid om de synchronisatieloga precies af te stemmen op de projectvereisten.
- Geen afhankelijkheid van externe diensten of abonnementen die kunnen veranderen of duurder worden.
- Eenvoudig uit te breiden naar andere platformen, zoals Shopify, zonder de volledige aanpak te moeten herzien.

n8n blijft beschikbaar als bijkomende tool voor visualisatie van processen of eenvoudige integraties, maar vormt niet de kern van de oplossing.

2.6. Keuze van programmeertaal: Python vs. C#

Naast de keuze van de aanpak werd ook onderzocht in welke programmeertaal het script het best geschreven wordt. De twee kandidaten waren Python en C# (.NET).

Uit onderzoek bleek dat de beschikbare .NET-bibliotheken voor Odoo-integratie grotendeels verouderd zijn en niet langer actief onderhouden worden. Dat brengt een risico met zich mee: de koppeling zou mogelijk regelmatig bijgewerkt of herschreven moeten worden bij updates van Odoo of de API.

Python heeft dat nadeel niet. De taal wordt actief gebruikt in de bredere Odoo-gemeenschap en wordt ook intern binnen het bedrijf gebruikt door andere medewerkers. Dit maakt de code beter overdraagbaar: collega's kunnen het script lezen, begrijpen en indien nodig aanpassen.

2.7. Besluit van de analyse

De analyse toont aan dat no-code tools zoals Zapier en n8n geschikt zijn voor eenvoudige of eenmalige koppelingen, maar tekortkomingen vertonen bij complexere synchronisatievereisten en hogere volumes. Een Python-script op maat sluit het beste aan bij de doelstellingen van dit project en biedt de meeste mogelijkheden voor de toekomst.

Op basis van deze analyse werd de implementatie aangevat. De volgende hoofdstukken beschrijven hoe de twee centrale functionaliteiten (productsynchronisatie en ordersynchronisatie) in de praktijk werden uitgewerkt.

3. Synchronisatie van producten

3.1. Productsynchronisatie: Odoo → WooCommerce

De productsynchronisatie werkt in één richting: Odoo is de centrale databron voor alle productinformatie en voorraadbeheer. Periodiek controleert het systeem welke producten actief zijn in Odoo en vergelijkt die met de producten die al in WooCommerce staan. De vergelijking gebeurt op basis van het Stock Keeping Unit (SKU)-nummer, een unieke code die elk product identificeert (Odoo B.V., z.d.-a).

***Wat is een SKU?** SKU staat voor Stock Keeping Unit. Het is een unieke identificatiecode voor elk product, vergelijkbaar met een artikelnummer. Dankzij de SKU weet het systeem precies welk product in Odoo overeenkomt met welk product in WooCommerce. Dit is ook de reden waarom een correcte SKU onmisbaar is voor een betrouwbare synchronisatie.*

Hoe het synchronisatieproces werkt

Het synchronisatieproces verloopt via vier vaste stappen (zie ook Bijlage B voor een schematisch overzicht):

- **Verbinding opzetten:** het script verbindt zich met zowel Odoo als WooCommerce via hun respectievelijke API's.
- **Producten ophalen:** alle actieve producten worden opgehaald uit Odoo, inclusief gegevens zoals naam, SKU, prijs, beschrijving en beschikbare voorraad.
- **Vergelijken:** het script bouwt een overzicht van alle reeds bestaande producten in WooCommerce op basis van hun SKU.
- **Bijwerken of aanmaken:** voor elk product wordt gecontroleerd of het al bestaat in WooCommerce. Zo ja, worden de gegevens bijgewerkt. Zo nee, wordt het product nieuw aangemaakt.

Onderstaande afbeelding toont de kern van de synchronisatielogica, zoals geïmplementeerd in het bestand main.py:

```
def _sync_single_product(product, woo, brand_map, sku_map, dry_run, summary):
    sku = (product.default_code or "").strip()
    if not sku:
        summary.skipped_no_sku += 1
        return

    payload = map_odoo_to_woo(product, brand_map)

    if sku in sku_map:
        woo.update_product(sku_map[sku], payload.model_dump(exclude_none=True))
        summary.updated += 1
    else:
        woo.create_product(payload.model_dump(exclude_none=True))
        summary.created += 1
```

Figuur 1. Kernlogica van de product synchronisatie in main.py.

Simulatiemodus (dry-run)

Voordat het script daadwerkelijk wijzigingen doorvoert, kan het uitgevoerd worden in simulatiemodus (ook wel dry-run genaamd). In deze modus toont het systeem een volledig overzicht van welke producten aangemaakt of bijgewerkt zouden worden, zonder iets effectief te wijzigen. Dit maakt het mogelijk om de werking te controleren voordat er iets in de productieomgeving verandert.

```
(.venv) PS C:\Users\Brent\Desktop\Vervio\api-center\WooCommerce> python main.py --products

[1/4] Verbinden met Odoo en WooCommerce...

15:07:21 INFO      Odoo versie gedetecteerd: 19.0-20260409 (major=19)
15:07:21 INFO      Odoo v19 – REST API (JSON-2) modus via API key.
15:07:21 INFO      Opgehaald WooCommerce config voor 1 bedrijf(ven)
15:07:21 INFO      Geladen 1 bedrijfsconfiguratie(s) uit Odoo
15:07:21 INFO      - My Company (interval=60 min stock tracking=disabled)
15:07:21 INFO      WooCommerce SKU's ophalen – pagina=1
15:07:21 INFO      WooCommerce SKU-map gebouwd – 79 SKU's gevonden

[2/4] Producten ophalen uit Odoo...

15:07:21 INFO      Producten ophalen...
15:07:22 INFO      Totaal 79 producten opgehaald uit Odoo

[3/4] Merken synchroniseren...

15:07:22 INFO      WooCommerce merken-map gebouwd – 1 merken
1 merk(en) beschikbaar in WooCommerce.

[4/4] Producten synchroniseren met WooCommerce...

15:07:22 INFO      WooCommerce product bijwerken – woo_id=7386 SKU=105007-5800-116
[UPDATE] SKU=105007-5800-116 Authentic Polysuit – Royal-Black (116)
15:07:23 INFO      WooCommerce product bijwerken – woo_id=7387 SKU=105007-5800-128
[UPDATE] SKU=105007-5800-128 Authentic Polysuit – Royal-Black (128)
15:07:25 INFO      WooCommerce product bijwerken – woo_id=7388 SKU=105007-5800-140
[UPDATE] SKU=105007-5800-140 Authentic Polysuit – Royal-Black (140)
```

Figuur 2. Terminaloverzicht bij het activeren van de simulatiemodus.

Bulkacties ▼ Toepassen Selecteer een categorie ▼ Op producttype filteren ▼ Filteren op voorraadstatus ▼ Filter op merk ▼ Filter						
☐	 Naam ⬆	SKU ⬆	Prijs ⬆	Categorieën	Tags	Merken
☐	 Aarhus Shirt – Royal-Black (128)	110004-5800-128	€ 24,99	Geen categorie	–	hummel
☐	 Aarhus Shirt – Royal-Black (116)	110004-5800-116	€ 24,99	Geen categorie	–	hummel
☐	 Aarhus Shirt – Shocking Orange-Black (XXXL)	110004-3840-XXXL	€ 26,99	Geen categorie	–	hummel
☐	 Aarhus Shirt – Shocking Orange-Black (XXL)	110004-3840-XXL	€ 26,99	Geen categorie	–	hummel
☐	 Aarhus Shirt – Shocking Orange-Black (XL)	110004-3840-XL	€ 26,99	Geen categorie	–	hummel
☐	 Aarhus Shirt – Shocking Orange-Black (S)	110004-3840-S	€ 26,99	Geen categorie	–	hummel
☐	 Aarhus Shirt – Shocking Orange-Black (M)	110004-3840-M	€ 26,99	Geen categorie	–	hummel
☐	 Aarhus Shirt – Shocking Orange-Black (L)	110004-3840-L	€ 26,99	Geen categorie	–	hummel

Figuur 3. Productlijst in Odoo na afloop van het synchronisatieproces.

3.2. Merken (brands)

WooCommerce ondersteunt productmerken via een afzonderlijke categorie. Om te vermijden dat merken handmatig ingevoerd moeten worden, verloopt dit proces volledig automatisch.

Vóór elke synchronisatie haalt het script alle reeds bestaande merken op uit WooCommerce en slaat die op in een intern overzicht. Wanneer een product in Odoo een merk heeft dat nog niet bestaat in WooCommerce, wordt dat merk automatisch aangemaakt (WordPress.org, z.d.). Op die manier zijn de merken steeds actueel zonder manuele tussenkomst.

4. Order terugkoppeling

4.1. Ordersynchronisatie: WooCommerce → Odoo

De ordersynchronisatie werkt in de omgekeerde richting van de productsynchronisatie.

Wanneer een klant een bestelling plaatst op de webshop, wordt die bestelling automatisch overgebracht naar Odoo als een verkoopofferte. Het systeem controleert periodiek of er nieuwe bestellingen zijn met de status 'in verwerking' (betaald, te verwerken) en verwerkt die één voor één (Odoo B.V., z.d.-c).

Dit proces verloopt in vijf stappen:

Stap 1 - Klant opzoeken of aanmaken

Voor elke nieuwe bestelling zoekt het systeem de klant op in Odoo op basis van e-mailadres. Als de klant al bestaat, wordt diens bestaande record gebruikt. Bestaat de klant nog niet in Odoo, dan wordt automatisch een nieuw klantprofiel aangemaakt op basis van de factuurgegevens uit de WooCommerce-bestelling.

Stap 2 - Voorkomen van dubbele invoer

Om te vermijden dat dezelfde bestelling meerdere keren ingevoerd wordt, controleert het systeem vóór elke verwerking of er al een offerte bestaat met het WooCommerce-bestelnummer. Zo wordt elke bestelling gegarandeerd slechts één keer verwerkt, zelfs als het script meerdere keren na elkaar uitgevoerd wordt.

Stap 3 - Verkoopofferte aanmaken in Odoo

De artikelen uit de webshop bestelling worden vertaald naar orderregels in Odoo. Elk artikel wordt opgezocht via zijn SKU-nummer. De eenheidsprijs wordt berekend op basis van het totaalbedrag, zodat eventuele kortingen automatisch correct verwerkt worden.

Stap 4 - Automatisch bevestigen van betaalde bestellingen

Als de bestelling in WooCommerce de status 'in behandeling' heeft, wat betekent dat de klant betaald heeft, wordt de verkoopofferte in Odoo automatisch bevestigd. Daardoor verandert de status van offerte naar definitieve verkooporder en kan de verdere verwerking en verzending starten.

Stap 5 - Terugkoppeling naar WooCommerce

Na de succesvolle aanmaak in Odoo wordt het Odoo-referentienummer (bijv. S-000456) teruggestuurd naar de WooCommerce-bestelling en daar opgeslagen. Op die manier is in de webshop meteen te zien dat de bestelling verwerkt is en welk intern nummer zij heeft gekregen in Odoo.

☐	#7506 Brent Paessens		29 apr 2026	In afwachting	✓ S00461	€ 49,98	Direct
☐	#7505 Brent Paessens		29 apr 2026	In behandeling	✓ S00460	€ 24,99	Direct

Figuur 4. Odoo-referentienummer toegevoegd aan een WooCommerce-bestelling (Eigen werk).

S00201

Klant	demo demo	Vervaldatum ?	3 jun
	Teststraat 5	Offertedatum ?	4 mei, 10:38
	9000 Gent	Prijslijst ?	Standaard (EUR)
	België	Betaalvoorwaarden	Onmiddellijk
	BE0123456749		

Orderregels Overige info Handtekening klant

Product	Hoeveelheid	Prijs	Btw	Bedrag
[110004-5800-128] Aarhus Shirt - Royal-Black (128)	1,00	19,99	21%	19,99 €
Aarhus Shirt - Royal-Black (128)				

Product toevoegen Sectie toevoegen Voeg een notitie toe Catalogus

WooCommerce Order #7526

	Korting
Excl. btw:	19,99 €
BTW 21%:	4,20 €
Totaal:	24,19 €

Figuur 5. Verkoopofferte in Odoo met verwijzing naar het WooCommerce-bestelnummer.

5. Planning

Onderstaande tabel geeft een overzicht van de weekplanning zoals die tijdens het project werd gevolgd. Kleine afwijkingen ten opzichte van de oorspronkelijke planning zijn normaal bij softwareontwikkeling. Waar nodig werden bijstellingen doorgevoerd na overleg met de stagebegeleider.

Periode	Activiteiten
week 1	Kennismaking met de werkomgeving. Experimenteren met WordPress, Odoo en externe tools.
week 2	Onderzoek naar product synchronisatie: vergelijking van externe tools en scriptoplossingen, uitgewerkt in een analysedocument.
week 3	Onderzoek naar orderterugkoppeling naar Odoo: vergelijking van aanpakken, uitgewerkt in een analysedocument.
week 4	Ontwikkeling van een Python-script voor het inladen van producten en het ophalen van bestellingen via de beschikbare API's.
week 5	Testen van de API-endpoints van WordPress en Odoo.
week 6	Verwerken van facturen van een externe klant; ontvangen van feedback op de huidige voortgang.
week 7	Onderzoek naar klantenbeheer in Odoo, verwerken van feedback in de scripts.
week 8	Verdere uitwerking van de scripts met focus op de WordPress-kant.
week 9	Verdere uitwerking van de scripts en documentatie.
week 10	Verdere uitwerking van de scripts en testen
week 11	Verdere uitwerking van de scripts, testfase en start van on-premise omgeving op te stellen.
week 12	Verdere uitwerking on-premise omgeving + testen
week 13	Afwerking van de on-premise en testen van live omgeving.

6. Besluit

Dit project had als doel een werkende koppeling te bouwen tussen Odoo en WooCommerce, zodat productgegevens automatisch gesynchroniseerd worden en bestellingen uit de webshop automatisch verwerkt worden in Odoo. Beide centrale functionaliteiten werden succesvol gerealiseerd.

Terugblik

De analyse in de beginfase wees uit dat no-code tools zoals Zapier en n8n niet de meest geschikte keuzes zijn voor dit project. Zij zijn handig voor eenvoudige koppelingen, maar tonen hun beperkingen zodra de synchronisatie logica complexer wordt. Via de Weighted Ranking Methode werd een onderbouwde keuze gemaakt voor een Python-script op maat, en die keuze heeft zich doorheen het project bewezen.

De product synchronisatie werkt op basis van SKU-vergelijking: Odoo fungeert als de enige bron van waarheid en WooCommerce wordt periodiek bijgewerkt. De order synchronisatie zorgt ervoor dat betaalde bestellingen automatisch als verkooporder worden aangemaakt in Odoo, compleet met klantbeheer, het vermijden van dubbele invoer en terugkoppeling van het Odoo-referentienummer naar WooCommerce.

Evaluatie ten opzichte van de doelstellingen

De vooropgestelde doelstellingen werden grotendeels gerealiseerd:

- De automatische product synchronisatie is operationeel en werd getest in een live omgeving.
- De orderverwerking werkt betrouwbaar dankzij ingebouwde controles voor het vermijden van duplicaten en klantbeheer.
- De oplossing is flexibel genoeg om later uitgebreid te worden naar andere platform o.a. Shopify.

Aanbevelingen voor de toekomst

De huidige implementatie biedt een stevige basis voor verdere ontwikkeling. één concrete aanbeveling:

- **Shopify-connector afronden:** de Shopify-koppeling volgt dezelfde architectuur als de WooCommerce-koppeling en is relatief eenvoudig te voltooien dankzij de gedeelde Python-basis.

Samenvattend kan gesteld worden dat dit project een waardevolle leerervaring was. Het heeft niet alleen geleid tot een werkende en productierijpe integratie, maar ook tot een dieper inzicht in de werking van ERP-systemen, API-communicatie en Python-ontwikkeling in een professionele context.

LITERATUURLIJST

Odoo. (z.d.-a). Odoo eLearning.

https://www.odoo.com/nl_NL/slides

Odoo. (z.d.-b). Odoo – Open source ERP en CRM.

https://www.odoo.com/nl_NL

Odoo. (z.d.-c). Odoo 19.0 documentatie.

<https://www.odoo.com/documentation/19.0/>

Odoo B.V. (z.d.-d). External JSON-2 API. *External JSON-2 API*.

https://www.odoo.com/documentation/19.0/developer/reference/external_api.html

WordPress. (z.d.). Using the REST API.

<https://developer.wordpress.org/rest-api/using-the-rest-api/>

WooCommerce. (z.d.). WooCommerce REST API. *WooCommerce REST API*.

<https://woocommerce.github.io/woocommerce-rest-api-docs/>

Cybrosys Technologies. (z.d.). *Cybrosys Technologies – YouTube-kanaal*.

<https://www.youtube.com/@cybrosys-technologies>

n8n. (z.d.). *n8n workflow automation platform*.

<https://n8n.io/>

Zapier. (z.d.). *Zapier automation platform*.

<https://zapier.com/>

BIJLAGEN

De bijlagen bevatten aanvullend materiaal waarnaar in de hoofdtekst verwezen wordt. Het document is ook zonder de bijlagen volledig begrijpelijk; de bijlagen dienen uitsluitend ter verdieping en ondersteuning.

Bijlage A - Weighted Ranking Methode (detailberekening)

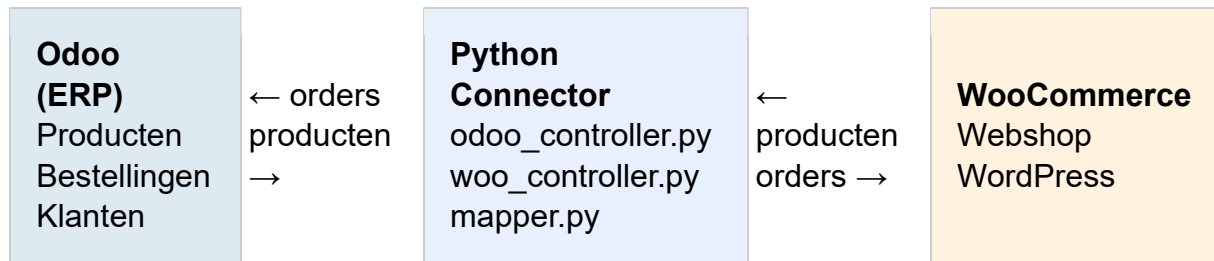
In hoofdstuk 2.4 werd de Weighted Ranking Methode toegepast om een objectieve toolkeuze te maken. Onderstaande tabel toont de volledige berekening: per criterium werd een gewicht (W) en een score per oplossing (S) toegekend. Het gewogen deelresultaat ($W \times S$) werd opgeteld tot een eindtotaal.

Criterium	Gewicht (W)	Zapier (S)	Zapier (W×S)	n8n (S)	n8n (W×S)	Python (S)	Python (W×S)
Kostprijs	30 %	2	0,60	2	0,60	5	1,50
Flexibiliteit	25 %	2	0,50	3	0,75	5	1,25
Schaalbaarheid	20 %	3	0,60	3	0,60	5	1,00
Gebruiksgemak	15 %	4	0,60	3	0,45	2	0,30
Techn. controle	10 %	1	0,10	3	0,30	5	0,50
Totaal	100 %		2,40		2,80		4,40

De Python-oplossing behaalt met 4,40 een duidelijk hogere score dan Zapier (2,40) en n8n (2,80), voornamelijk door de maximale score op kostprijs, flexibiliteit, schaalbaarheid en technische controle.

Bijlage B - Projectarchitectuur

Onderstaand schema (Figuur 6) geeft een vereenvoudigd overzicht van de architectuur van het synchronisatiesysteem zoals beschreven in de hoofdstukken 3 en 4.



Figuur 6. Vereenvoudigd architectuuroverzicht van het synchronisatiesysteem (Eigen werk).

Bijlage C – Kern code fragmenten

Onderstaande fragmenten illustreren twee centrale functies waarnaar verwezen wordt in de hoofdstukken 3 en 4. De volledige broncode is beschikbaar in de GitHub-repository van het project.

Fragment 1 – Product synchronisatie: aanmaken of bijwerken (main.py)

Dit fragment toont de kern logica van de product synchronisatie. Op basis van de bestaande SKU-map wordt bepaald of een product aangemaakt of bijgewerkt moet worden:

```
# Bepaal actie op basis van bestaande SKU-map
if sku in sku_map:
    woo.update_product(sku_map[sku], payload_dict)    # bestaand product bijwerken
    summary.updated += 1
else:
    woo.create_product(payload_dict)                  # nieuw product aanmaken
    summary.created += 1
```

Fragment 2 – Duplicatiecontrole vóór orderaanmaak (odoo_controller.py)

Voor elke bestelling wordt gecontroleerd of er al een verkoopofferte bestaat met het WooCommerce-bestelnummer, om dubbele invoer te vermijden:

```
def order_exists(self, woo_order_number: str, company_id: int | None = None) -> bool:
    note_value = f"WooCommerce Order #{woo_order_number}"
    domain = [{"note", "ilike", note_value}]
    if company_id:
        domain.append(["company_id", "=", company_id])
    ids = self._json2_search("sale.order", domain)
    return bool(ids)    # True = bestelling bestaat al, overslaan
```

Bijlage D – Lijst van gebruikte afkortingen

Onderstaande tabel geeft een overzicht van alle afkortingen die in dit document worden gebruikt. Bij de eerste vermelding in de tekst wordt elke afkorting voluit geschreven, gevolgd door de afkorting tussen haakjes.

Afkorting	Volledige term	Eerste vermelding
API	Application Programming Interface	Hoofdstuk 2.2
ERP	Enterprise Resource Planning	Hoofdstuk 1
kmo	Kleine en middelgrote onderneming	Hoofdstuk 1
SKU	Stock Keeping Unit	Hoofdstuk 3.1